

CinemaSeat — architecture & data pipeline

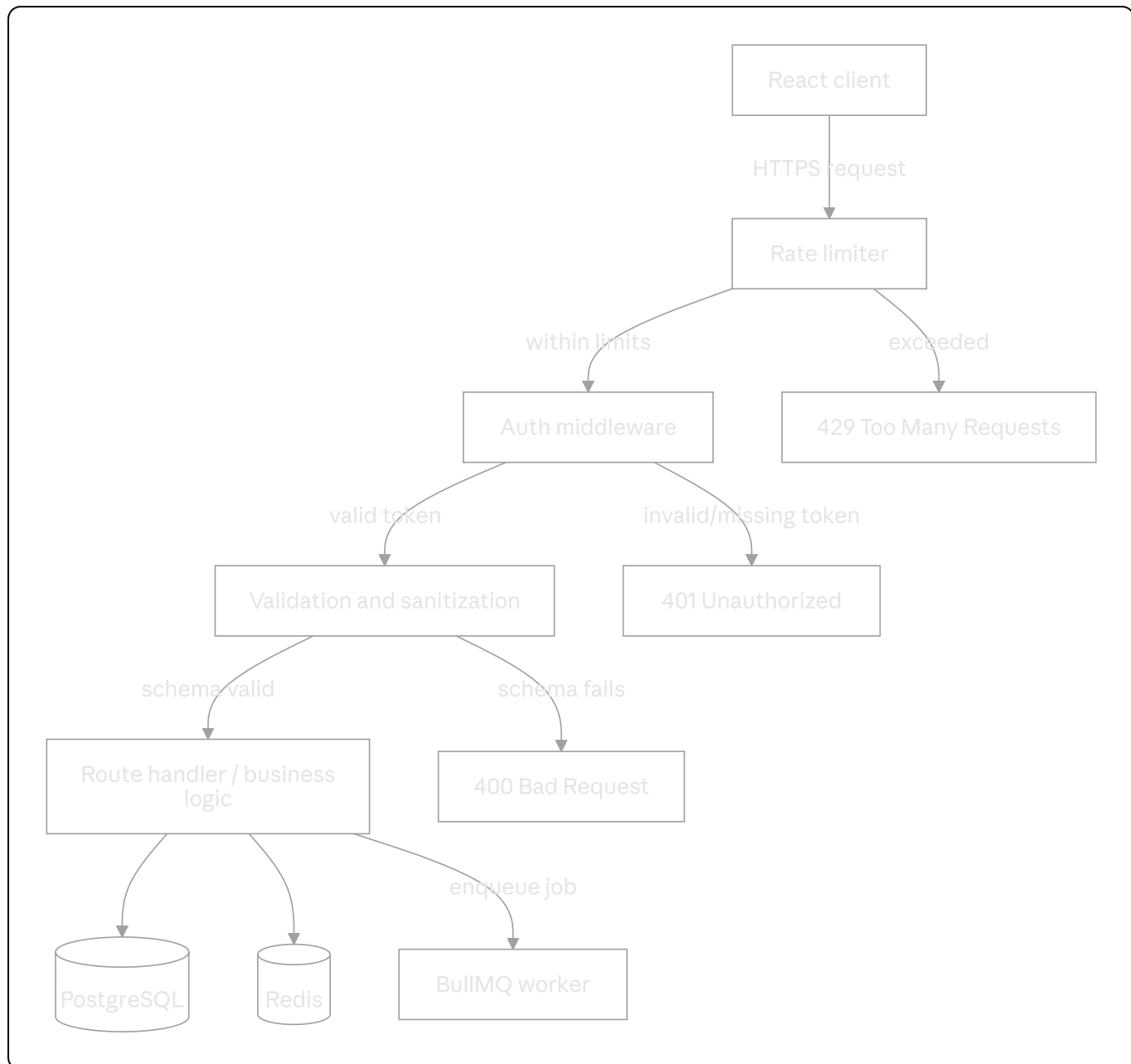
This document explains what the system does at each stage, how it's implemented, and why that choice was made over the alternatives. It's written to double as the architecture section of the hackathon README and as a reference for the team while building.

Stack: React (frontend) · Node.js / Express modular monolith (backend) · PostgreSQL (source of truth) · Redis (locks, OTP, rate limits, job queue) · BullMQ (background workers) · provided mock payment/OTP gateway.

0. The one idea that repeats everywhere

Reject cheap, reject early. Every layer of this system is ordered so that the least expensive check happens first, and a request only reaches the expensive, contended resource (Postgres) after it has survived everything cheaper. You'll see this pattern three times: the middleware stack (rate limit → auth → validation, before any handler logic runs), the seat-hold flow (Redis lock before Postgres transaction), and the webhook handler (idempotency check before any business logic). It's the same principle applied at three different layers, and it's worth stating explicitly in DECISIONS.md — it's the reasoning that ties the whole design together.

1. Request lifecycle — the full path of one HTTP request



What: every request passes through the same ordered middleware chain before it ever reaches a handler. **How:** `helmet` (security headers) → `cors` → rate limiter (Redis-backed) → body parser with a size cap → JWT auth check → route-specific schema validation/sanitization → handler. **Why this order specifically:** each step is cheaper than the one after it. A rate-limited or unauthenticated request is rejected in microseconds without ever touching a validation schema or the database. Doing validation before auth, or auth before rate limiting, would mean paying the cost of the more expensive check for traffic you were always going to reject anyway.

2. Frontend validation

What: format and completeness checks on forms — email/phone format, required fields, seat selection limits, OTP code shape — before a request is even sent.

How: schema-based validation (Zod or Yup) wired into the form layer (React Hook Form), disabling submit until the schema passes. The *same* schema shape is mirrored on the backend so the two never drift apart silently.

Why: this is a UX layer, not a security layer. Its entire job is to give the user instant feedback and avoid wasting a round trip on a request that was always going to fail — it has zero bearing on data integrity. Anyone can bypass client-side validation with a raw HTTP call, so it must never be the thing standing between bad data and your database. That job belongs entirely to the backend, which is why the same schema exists on both sides but only one of them is trusted.

3. Backend validation and sanitization

What: the real trust boundary. Every field is re-validated server-side regardless of what the client already checked, and free-text fields are sanitized before they touch the database or get rendered anywhere.

How:

- Schema validation as Express middleware, per route, using the same Zod schema shape as the frontend — reject with `400` and a field-level error list before the handler runs.
- All database access goes through parameterized queries (via `pg` or an ORM's query builder) — never string-concatenated SQL. This is what actually prevents SQL injection; sanitization libraries are a second layer, not a substitute for parameterization.
- Free-text fields (if any — e.g. a display name) get trimmed and escaped with `validator.js` before storage.
- Money is stored and computed as integer cents (`amount_cents: 45000`), not `450.00`), never as a float — floating-point arithmetic on currency is a real, avoidable source of accounting drift.

Why: the client cannot be trusted, full stop — not because your users are malicious by default, but because the request never has to come from your frontend at all. The backend schema is the actual gate; the frontend one is a convenience copy of it.

4. Concurrency control — the seat-hold path

This is the highest-stakes write in the system, so it gets two layers with distinct jobs.

```
sequenceDiagram
    participant A as User A
    participant B as User B
    participant R as Redis
    participant P as Postgres
    A->>R: SET seat:F12 NX EX ttl
    B->>R: SET seat:F12 NX EX ttl
    R-->>A: OK (lock acquired)
    R-->>B: fail - key exists
    Note over B: rejected in <1ms, no DB hit
    A->>P: BEGIN; SELECT status FOR UPDATE
    P-->>A: status = available
    A->>P: UPDATE status='held'; INSERT hold; COMMIT
    P-->>A: 200 - seat held
```

What: exactly one of N concurrent requests for the same seat succeeds; all others are rejected cleanly with no partial state.

How — two layers, different responsibilities:

1. **Redis** (`SET key NX EX <hold_ttl>`) — an atomic, sub-millisecond check. This is a *performance* filter: under a burst of 100 requests for one seat, it rejects 99 of them before they ever reach the database, so Postgres only ever sees the request(s) that actually have a shot.
2. **Postgres** (`SELECT ... FOR UPDATE`) **inside a transaction** — the *correctness* guarantee. This locks the specific `(showtime_id, seat_id)` row; any second transaction targeting the same row blocks until the first commits, then sees the updated status and is rejected. This holds even if Redis were ever wrong — a restart, an eviction, a bug — because the database constraint doesn't depend on Redis being correct.

Why two layers and not one: Redis alone is fast but not durable — nothing stops two requests from both believing they hold the lock if Redis misbehaves. Postgres alone is correct but means every one of the 100 requests queues on the same row lock, which is fine for a single seat but would strain the database under a genuinely large simultaneous rush across many seats. Together, Redis absorbs the burst and Postgres guarantees correctness even in Redis's worst case.

Hold expiry: lazy expiry on read (`(hold_expires_at < now())` → treat as available) plus a BullMQ repeatable job sweeping expired holds back to `(available)` every few seconds. Redis's own key TTL expires in parallel, so a new `SETNX` on that seat naturally succeeds again the moment the hold lapses — no manual coordination between the two expiries is needed.

5. Data integrity guarantees

Mechanism	What it protects against
<code>(showtime_id, seat_id)</code> composite primary key	Two rows ever representing the same seat on the same showtime
<code>SELECT ... FOR UPDATE</code> transaction	Two bookings confirmed for one seat, even under concurrent writers
<code>webhook_events.event_id</code> unique constraint	A duplicate gateway callback creating a second payment or confirming a booking twice
Integer cents for all money fields	Floating-point rounding drift in stored/reported revenue
Single transaction for "confirm booking + mark seat booked"	A seat marked booked with no corresponding booking row, or vice versa, if the process crashes mid-write
<code>created_at</code> / <code>updated_at</code> timestamps on bookings and payments	No audit trail to reconstruct what happened during a disputed or duplicate transaction

Why a table: each of these is a single, boring mechanism doing one job. None of them requires custom application logic to reason about — the guarantee comes from the database itself, which is more trustworthy than any amount of "we checked for this in the handler" code.

6. Payment — the async path

What: `/pay` returns almost immediately; the actual outcome (success, failure, duplicate) arrives later via a webhook the gateway calls back on.

How:

1. Handler calls the gateway's `/charge` endpoint and returns `202 Pending` to the client right away — it does **not** wait for the gateway, because the gateway's callback is documented as arriving 2–15 seconds later.
2. The gateway calls `POST /webhooks/gateway` with the outcome. That handler does this, in order:

sql

```
INSERT INTO webhook_events (event_id, payment_id, received_at)
VALUES ($1, $2, now())
ON CONFLICT (event_id) DO NOTHING
RETURNING event_id;
```

No row returned → this `event_id` was already processed → return `200` and stop, doing nothing further. Row returned → first time seeing it → update `payments.status` and confirm the `booking` in one transaction. 3. The handler always returns `200`, even for a duplicate — returning anything else tells the gateway delivery failed, and it retries forever per the spec.

Why: the 8% duplicate-callback rate and 2-15 second delay aren't edge cases here, they're the documented normal behavior of the gateway. Building the payment/booking confirmation entirely around "the callback might arrive twice or late, and my handler must be safe either way" means none of that behavior is a special case to handle — it's just the expected shape of every request.

7. Background processing

What: two jobs run outside the request/response cycle — sweeping expired holds, and processing gateway webhooks that need more than a quick idempotency check.

How: BullMQ, backed by the same Redis instance (separate key namespace, `bull:*`), running as a second process from the same codebase (`node worker.js` vs `node server.js`), both in the same Docker image.

Why a separate process instead of doing this inline in the request handler: isolation. A slow or flaky gateway call recovering from a timeout shouldn't block or slow down someone browsing showtimes. Running it as its own process means the worst-case gateway misbehavior degrades payment processing latency, not the rest of the app's responsiveness — you get part of the benefit people usually reach for microservices for, without actually needing separate services.

8. Keeping the UI honest

What: the person who won the seat sees it confirmed instantly; everyone else looking at the same seat map sees it flip to held/booked without needing a manual refresh.

How:

- The acting user's own client updates optimistically the moment the hold request returns `200` — it doesn't wait for anything else, since it already has the authoritative answer.
- Everyone else's seat map polls the showtime's seat state every few seconds (a plain `GET` against the same read path used for the initial page load — no new

infrastructure).

Why polling over WebSockets here: a WebSocket/SSE layer gives you instant push updates but adds real implementation and testing surface for an 8-hour build. Short polling is honest about the same information with a few seconds of staleness for passive viewers, while the person who actually competed for the seat — the one where correctness matters — never depends on polling at all, since their own request response is authoritative and immediate. If time allows after the required milestones are solid, upgrading the seat map to SSE is a clean, isolated follow-up.

Quick reference

Step	What	How	Why
Request entry	Reject bad traffic early	Rate limit → auth → validate, in that order	Cheapest checks first
Frontend validation	Format/completeness feedback	Zod schema in the form layer	UX speed, not security
Backend validation	The real gate	Same schema, server-side, parameterized queries	Client can't be trusted
Seat hold	Exactly one winner	Redis <code>SETNX</code> fast filter + Postgres <code>FOR UPDATE</code>	Speed + durability, together
Hold expiry	Seats return to the pool	Lazy expiry + BullMQ sweep, Redis TTL in parallel	No manual coordination needed
Payment	Never double-confirm	Fast <code>202</code> + idempotent webhook via unique <code>event_id</code>	Gateway is documented as unreliable
Background jobs	Isolate slow work	Separate BullMQ worker process, same Redis	One flaky call can't slow the whole app
UI freshness	Reflect the real state	Optimistic update for the actor, polling for viewers	Correctness doesn't depend on push infra